

Access-Adaptive Priority Search Tree

Sally Student* and Paul Professor
Anonymous University

Abstract

In this paper we introduce the notion of explicit worst-case bounded adaptive algorithms for applications with fixed process-completion requirements. Such applications demand that a process be guaranteed to complete within an established time interval while minimizing computational overhead during that interval, e.g., so as to minimize total energy usage. Our principal contribution is the *access-adaptive priority search tree* (AAPST), which can provide efficient distribution-sensitive performance comparable to the splay tree, but do so within strict – and $O(\log n)$ optimal – worst-case per-query bounds. More specifically, while the splay tree is conjectured to offer optimal adaptive amortized query complexity, it may require $O(n)$ for individual queries, whereas the AAPST offers competitive distribution-sensitive performance with strict $O(\log n)$ time complexity. This makes the AAPST more suitable for certain interactive (e.g., online and real-time) applications such as space system modules with reliability constraints involving rigid process-completion time intervals with secondary energy-minimization incentives.

Keywords: search trees, adaptive data structures, adaptive search trees, priority search tree, splay tree.

*This work was initiated while Ms. Student was an undergraduate research assistant.

1 Introduction

Complex systems with strict reliability requirements (e.g., as defined by specified safety or performance constraints) may impose rigid completion windows on some or all processes. For example, a given process is allotted t units of time to complete, and failure to meet its deadline is treated as a fault condition. Such a deadline is commonly defined to mitigate the impact of a divergent process on overall system performance by providing an unambiguous criterion for the signaling and subsequent handling of a fault condition. Of course, the window for a given process must be confidently established so as to avoid spurious interrupts. The study of such systems became a priority in the early space program [3] and are referred to as *time-critical tasks*:

A time-critical task is characterized by repetitions of C units of processing over a frame interval F within a deadline time $D \leq F$... The deadline, within which all processing must be completed, may result from a particular system implementation; that is, the output data must be available by D so that the system will have time to transfer the data out by the end of the frame. The deadline may also result from the requirements of the application itself. [2]

In many practical contexts, two different algorithms may be assessed as having equivalent utility if each can perform a particular operation within the same specified completion window, i.e., either algorithm is suitable for satisfying system requirements. However, in more stressing contexts in which energy consumption and/or heat generation must be minimized, e.g., for long-duration deep-space systems, there is need to optimize a more complex utility function. When evaluating prospective algorithms for such applications, there is typically a hard performance constraint that must be satisfied, plus a set of additional performance attributes that are *preferred* but not required. We refer to the latter as *augmented complexity considerations*.

In this paper we present an efficient adaptive-access data structure and associated retrieval algorithm that can simultaneously provide worst-case optimal query complexity while also adaptively reducing computational costs – which equates to minimizing energy consumption and heat generation – compared to prior state-of-the-art alternatives under various operating assumptions. Specifically, we introduce the access-adaptive priority search tree (AAPST), which is based on the priority search tree of McCreight [4], but with priorities representing key-access counts. We then demonstrate that our novel search algorithm offers strict optimal per-query performance bounds that cannot be supported using the state-of-the-art adaptive search tree, the splay tree [6], while demanding less computational overhead on average than a balanced binary search tree (BST) in practical contexts in which the access distribution of keys is highly nonuniform.

2 Background

In most practical applications, the access of information, e.g., keys from a large dataset, is highly non-random. In other words, some keys will be queried much more frequently than others. This nonuniform sampling from a dataset of n keys can potentially be exploited by a distribution-sensitive search structure to perform many fewer comparisons per query than required in the uniform case. For example, applications in which access frequencies are exponentially distributed, i.e., a small number of keys are accessed much more frequently than the remaining keys combined, an adaptive search algorithm and data structure may exhibit practical query performance that is much better than its $O(\log(n))$ complexity might suggest.

The splay tree [6] is a self-adjusting [1] binary search tree that optimizes its structure to the distribution patterns of the dataset. Splay trees differ from standard balanced BSTs by performing rotations that migrate frequently-accessed keys to the top of the tree so the search paths to those keys will be shorter when accessed during future queries. A novelty of the splay tree is that it does not necessarily enforce balance at all times during a given sequence of n updates and/or queries, but it does guarantee that the complexity of any given sequence is $O(n \log n)$. The value of the splay tree as an access-sensitive search structure is that it can offer sequence time complexity approaching $O(n)$ if the access distribution of keys is highly nonuniform. By contrast, a standard balanced BST (e.g., AVL, red-black, etc. [5]) provides no access-distribution sensitivity and thus can be expected to require $O(n \log n)$ time to perform a sequence of $O(n)$ operations.

A natural question is whether it is possible to combine the access-sensitive properties of the splay tree with the efficient worst-case properties of a balanced BST so that an $O(\log n)$ deadline time interval can be enforced while also minimizing the number of computations performed within that interval. In the next section we show that the *priority search tree* [4], which was introduced in the same year as the splay tree (1985), can be repurposed in the form of an *access-adaptive* priority search tree (AAPST) to simultaneously exploit nonuniform key accesses while enforcing an $O(\log n)$ process-completion time bound.

3 Adaptive Priority Search Tree

The *priority search tree* (PST) is a data structure introduced by Edward McCreight in 1985 with the objective of storing a set of n points in $\mathbb{R}^2$ in a way that allows for $O(\log n)$ update complexity, i.e., insertion or deletion of a point, and $O(\log n + k)$ complexity for semi-infinite 2-dimensional range queries where k is the number of returned objects. This complexity is achieved by maintaining the points simultaneously in BST order on the x coordinates and heap order on the y coordinates within the same binary tree structure. The data structure allows for five main operations on a dataset D of ordered pairs to be performed efficiently:

1. Insert an ordered pair into D .
2. Delete an ordered pair from D .
3. Given integers x_0 , x_1 , and y_1 , among all the pairs (x, y) in D such that $x_0 \leq x \leq x_1$ and $y \leq y_1$, find the pair whose x is minimal.
4. Given integers x_0 and x_1 , among all the pairs (x, y) in D such that $x_0 \leq x \leq x_1$, find the pair whose y is minimal.
5. Given integers x_0 , x_1 , and y_1 , enumerate all k pairs (x, y) in D such that $x_0 \leq x \leq x_1$ and $y \leq y_1$.

The priority search tree was the first data structure to support 2-dimensional spatial search queries within the same $O(\log n + k)$ complexity of 1-dimensional range queries offered by balanced binary search trees (BSTs) while also supporting $O(\log n)$ update operations. Specifically operations 1-4 have time complexity $O(\log n)$ and operation 5 has $O(\log n + k)$ complexity.

In his paper, Edward McCreight introduces two varieties of priority search trees, the first of which is the radix priority search tree. Each node in a radix PST contains exactly one ordered pair (x, y) . A radix PST is constructed so that the y -value of every child node is less than or equal to that of its parent in the case of a maximum PST, and greater than or equal to in the case of a minimum PST. In both cases, the x -value of every node in a right subtree is strictly less than that of every node in the left subtree. Furthermore, the cardinality of a node's right subtree is equal to or one less than the node's left subtree, ensuring balance. Unfortunately, this type of PST is not well suited for restructuring as a change to a singular node's y -value could require reconstruction of the entire tree in order to maintain its fidelity conditions. However, the second type of priority search tree, the balanced priority search tree, provides a solution to this issue. Unlike a radix PST node, a balanced PST node can contain up to two ordered pairs, the Q pair and the P pair. The Q pair (Q_x, Q_y) is required for every node and it is selected for its near-median x -value, similar to a standard balanced binary tree. The P pair (P_x, P_y) is optional and is chosen for its minimal y -value in the case of a minimum PST and maximal y -value in the case of a maximum PST, similar to a heap. Each pair (x, y) in the set of keys appears as the Q pair of exactly one node t , and may also appear as the P pair of at most one ancestor node of t . This structure allows for more efficient updating of a node's y -value.

While the priority search tree was originally created to store two-dimensional coordinates, we introduce here an alternative use of the data structure. With some slight construction and operation alterations, the balanced priority search tree can be used as a distribution-sensitive search structure. We will call this specialized structure an *access adaptive priority search tree* (AAPST). The AAPST stores each key of a dataset in the Q_x -value of a node and its respective access frequency as the Q_y -value. Additionally, a key and its respective access frequency may be stored in the P_x and P_y fields of an ancestor node such that the non-null P_y values form a max heap.

AOPST Node		
Field	Data Type	Description
Q x	Comparable	Key selected for near-median value of subtree
Q y	Integer	Priority associated with Q x
P x	Comparable	Key associated with maximal y-value in subtree
P y	Integer	Maximal priority in subtree
Valid P	Boolean	Whether the P-pair is valid
Left	AOPST Node	Left child
Right	AOPST Node	Right child
Parent	AOPST Node	Parent node

In other words, the skeleton of the tree maintains a balanced BST ordering of the keys while the access-frequencies associated with the keys are maintained in heap order. By slightly altering the search algorithm of a regular priority search tree, any search key can be found (or not found) in an AAPST in $O(\log n)$ time. The principal change to the standard balanced PST is the incrementing of the priorities (access frequencies) associated with the keys. Specifically, when a key is accessed by either an update or a query, its access count is incremented and the key's position in the heap may then be updated. More specifically, a modified access/query algorithm can be defined as follows:

1. Find the query key and increment its associated priority.
2. If the query key was found in the P field of a node, and the incremented priority does not exceed the P_y of its parent node, then return. If the incremented priority does exceed the P_y of its parent node, then delete the P pair and reinsert it to the proper position.
3. If the query key was found in the Q field of a node, insert the Q pair as a P pair in the proper position.

Step 1 takes time proportional to the pair's depth in the tree, and this will be the complexity of the operation in all cases in which the updated priority does not affect the heap order; otherwise, the complexity of the operation will be dominated by the $O(\log n)$ complexities of the P pair delete and insert algorithms. This establishes the worst-case $O(\log n)$ complexity of the new adaptive query algorithm.

Figures 1 through 8 visualize and AAPST and the effects this modified algorithm has on it as a set of keys is queried. The figure captions also note the number of comparisons required by each phase of the query process.

In the next section we provide practical comparisons of the AAPST and splay tree in the form of simulation results with varying degrees of non-uniformity in the sampling of query keys.

4 Comparative Performance Results

In this section we examine the relative performance characteristics of a conventional balanced binary search tree (BST), a splay tree, and the AAPST. In the

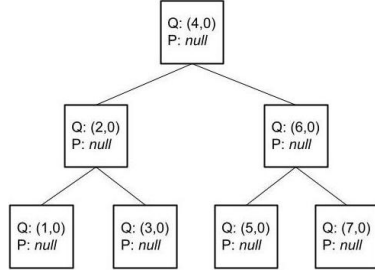


Figure 1: A visual representation of an Access-Adaptive Priority Search Tree before any queries have been completed.

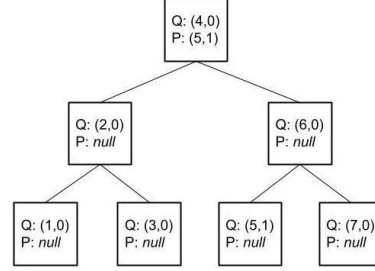


Figure 2: The AAPST from 1 after it is queried for the number 5. This query required 3 comparisons to find the key, for a total operation cost of 3 comparisons.

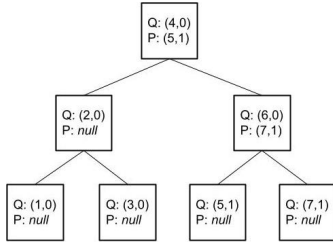


Figure 3: The AAPST from 2 after it is queried for the number 7. This query required 4 comparisons to find the key and one comparison to insert the found key and its access count as a P pair, for a total operation cost of 5 comparisons.

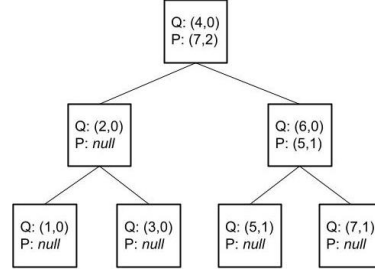


Figure 4: The AAPST from 3 after it is queried for the number 7. This query required 3 comparisons to find the key, one comparison to delete the p pair, and one comparison to reinsert the found key and its access count as a P pair, for a total operation cost of 5 comparisons.

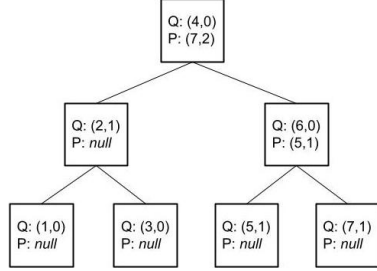


Figure 5: The AAPST from 4 after it is queried for the number 2. This query required 3 comparisons to find the key and one comparison to attempt to insert the found key and its access count as a P pair, for a total operation cost of 4 comparisons.

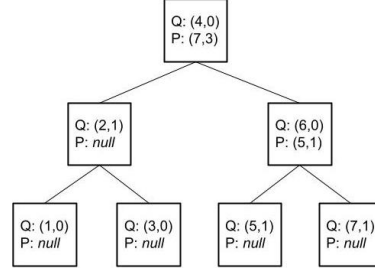


Figure 6: The AAPST from 5 after it is queried for the number 7. This query required 1 comparison to find the key, for a total operation cost of 1 comparison

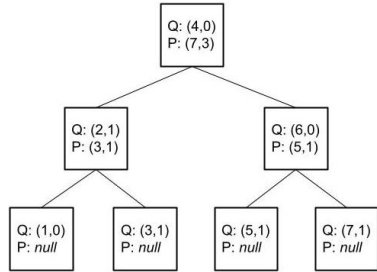


Figure 7: The AAPST from 6 after it is queried for the number 3. This query required 4 comparisons to find the key and one comparison to insert the found key and its access count as a P pair.

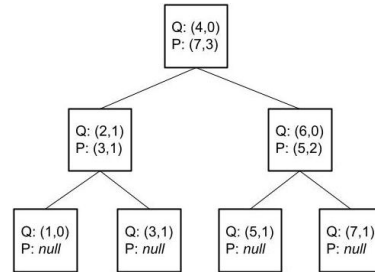


Figure 8: The AAPST from 7 after it is queried for the number 5. This query required 3 comparisons to find the key and one comparison to attempt to reinsert the found key and its access count as a P pair.

case of uniformly sampled query keys, the splay tree and AAPST incur extra overhead compared to the BST. In the case of the splay tree, this overhead takes the form of extra comparisons performed as the tree is restructured. In the case of the AAPST, the overhead takes the form of extra comparisons to delete and insert the P pair as well as an extra key comparison per node visited: one comparison to the key in the P_x position, and another comparison to the Q_x key if it was not found in the P pair. Therefore, the goal of our tests is to examine how relative number of key comparisons used during the search of each data structure is affected by the distribution of queried keys. We should expect the BST to be superior in the uniform case while the splay tree and AAPST should perform better as the distribution becomes increasingly nonuniform.

We define a value p , $0 \leq p \leq 1$, to parameterize our key-access testing distributions with $p = 0$ representing a uniform random distribution of key accesses; $p = 1$ representing an exponentially-distributed sequence of accesses with the most frequently-accessed key representing approximately 50% of the accesses, the next representing 25% of the accesses, etc., such that $O(\log n)$ of the keys comprise $O(n)$ of the accesses; and $0 < p < 1$ representing a weighted mixture of accesses from the two distributions.

As can be seen in Figure 9, the splay and AAPS trees perform comparably but are outperformed by the BST because of its lower overhead. In other words, the overhead of adaptivity incurred by the splay and AAPS trees does not yield dividends in the case of keys that are queried uniform-randomly. Figure 10 shows that the relative performance advantage of the BST decreases when 25% of the key accesses exponentially distributed access frequencies. Figure 11 shows that when there is an equal mix of keys sampled from the uniform and exponential distributions the three search structures perform comparably. In the case of all keys sampled exponentially, Figures 12 and 13 show that the distribution-sensitivity properties of the splay and AAPS trees provide a significant performance advantage over the BST as the access frequencies tend toward an exponential distribution.

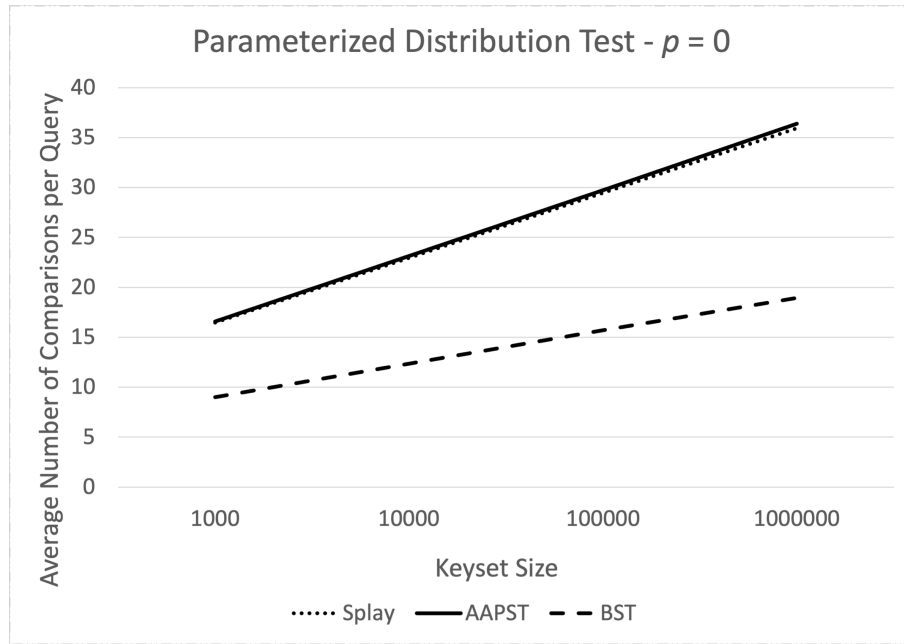


Figure 9: This figure shows the average number of key comparisons for query keys drawn from uniform distribution for datasets of increasing size n . The expected number of key comparisons performed by the BST is $\log(n)$ while the splay and AAPST trees perform roughly twice as many comparisons per query.

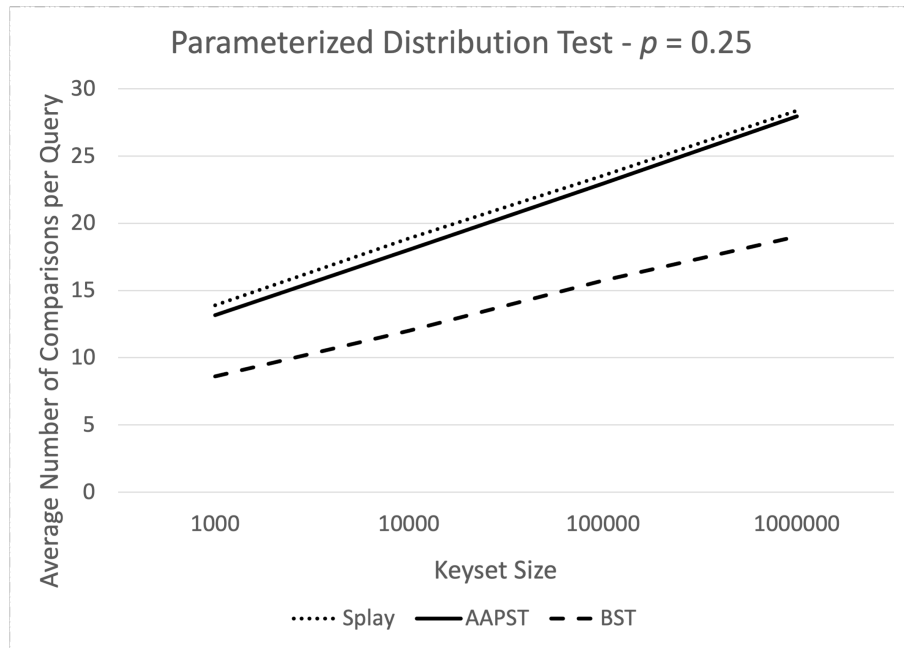


Figure 10: This figure shows the average number of key comparisons when the 1/4 of the query keys are sampled with exponential frequency and the remaining are sampled uniformly.

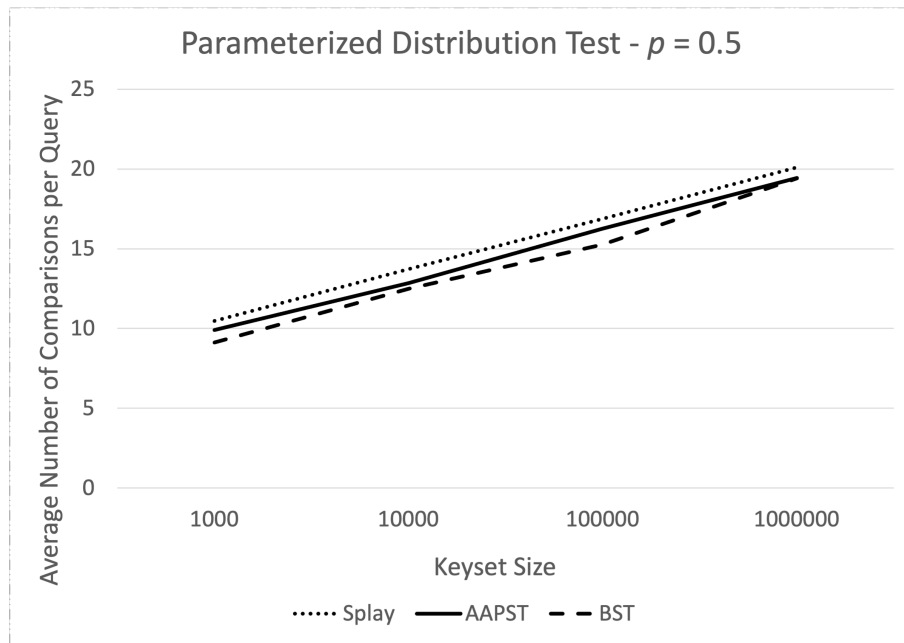


Figure 11: This figure shows the average number of key comparisons when half of the query keys are sampled with exponential frequency and the remaining half are sampled uniformly.

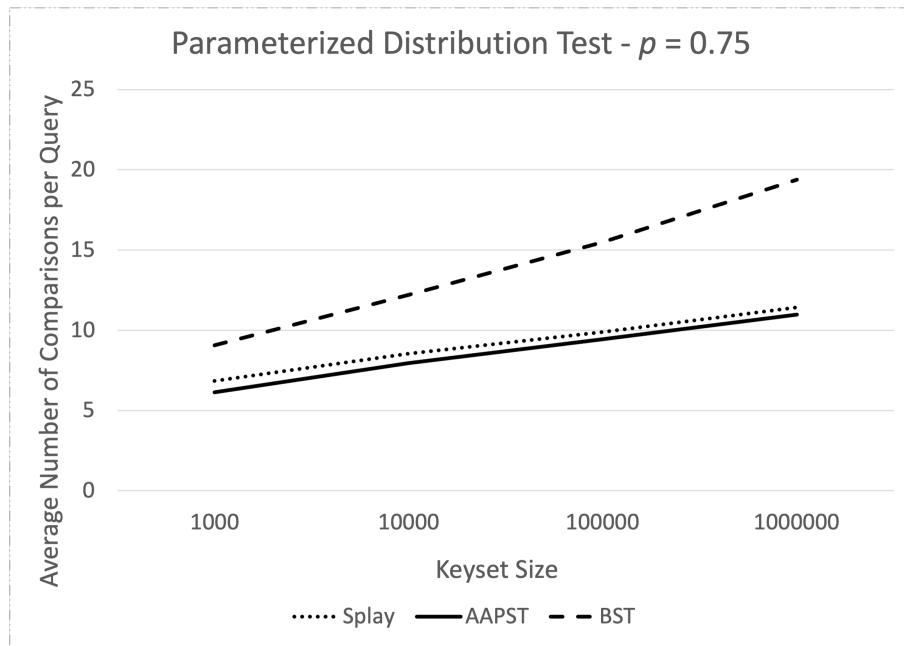


Figure 12: This figure shows the average number of key comparisons when 75% of the query keys are sampled according to an exponential distribution.

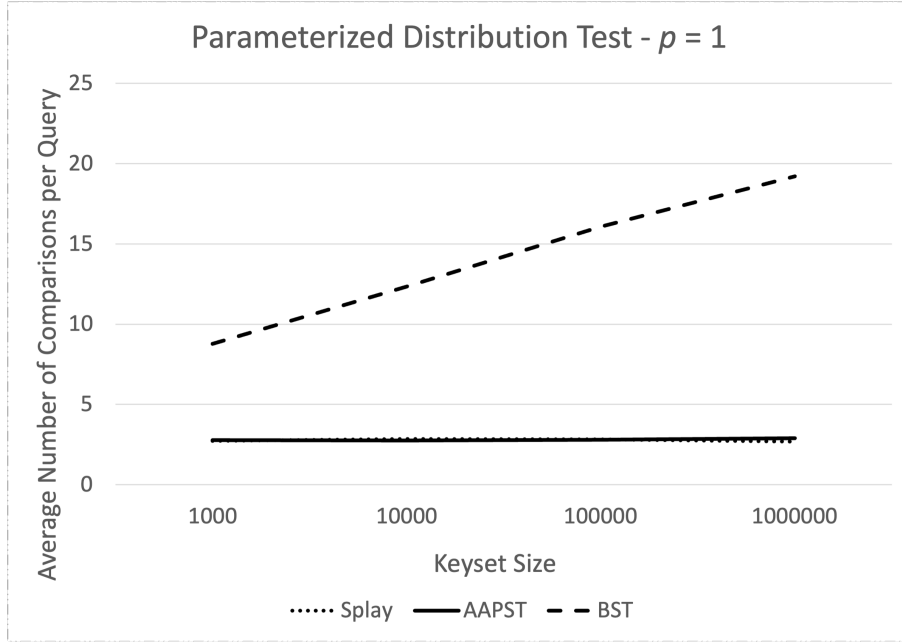


Figure 13: This figure shows the average number of key comparisons when all query keys are sampled according to an exponential distribution, i.e., the frequency of access of different keys decreases exponentially.

The test results presented in this section have focused on characterizing the relative performance of the three data structures in terms of a few key parameters. In the next section we present test results that are specific to applications involving a fixed process-completion time interval.

5 Test with Process-Completion Time Interval

The results of the previous section corroborate what should be expected: adaptivity provides increasing efficiency benefits as the key-access distribution deviates increasingly from uniform-random sampling. At $p = 0.75$, AAPST and Splay significantly outperform a static balanced BST, with convergence of the relative performance of Splay and AAPST as p approaches 1. In this section we focus on the principal use case for AAPST, which is to provide adaptive efficiency benefits with worst-case bounds for online and real-time applications.

Given the scenario of the previous section for a keyset of size of $100k$, the worst-case number of comparisons per query of a balanced BST can be determined *a priori* to be $\log_2(100k) = 17$. The worst-case for AAPST is $4\log_2(100k) - 2 = 66$, which derives from the fact that $2\log_2(n) - 1$ comparisons are needed to find a given key, and an additional $2\log_2(n) - 1$ comparisons

may be performed when the key is deleted and reinserted with its incremented priority.

Figure 14 plots the maximum number of comparisons required during each of consecutive sequence of 5 million queries. As should be expected, BST and AAPST each encounters a query during each interval that reaches its known upper bound number of key comparisons. The amortized complexity of Splay does not generally permit an $o(n)$ bound to be determined, so its worst-case query performance per sequence is highly variable and resembles that of a univariate normal distribution. This suggests that the maximum observed number of comparisons across all sequences would tend to increase with the number of sequences up to the trivial upper bound of n .

Figure 15 plots the average number of comparisons per sequence of 5 million queries. The first notable feature is the relatively slower rate at which AAPST initially adapts to the distribution. This is due to the fact that the AAPST is initialized with all priorities equal to zero, which means that each key is at random depth in the tree. This leads to many more comparisons per query until frequently-accessed keys migrate to their steady-state depth near the top of the tree. The second notable feature of Figure 15 is that its steady-state average number of comparisons for AAPST is one comparison per query higher than that of Splay¹. In summary, the test scenario of Figure 14 provides a concrete example of how the AAPST can provide superior average-case performance bounds while simultaneously enforcing a strictly-defined worst-case optimal upper bound.

¹Future work will examine whether the separate find and priority-update (delete and reinsert) steps of the AAPST search algorithm can be integrated to potentially eliminate a redundant comparison per node.

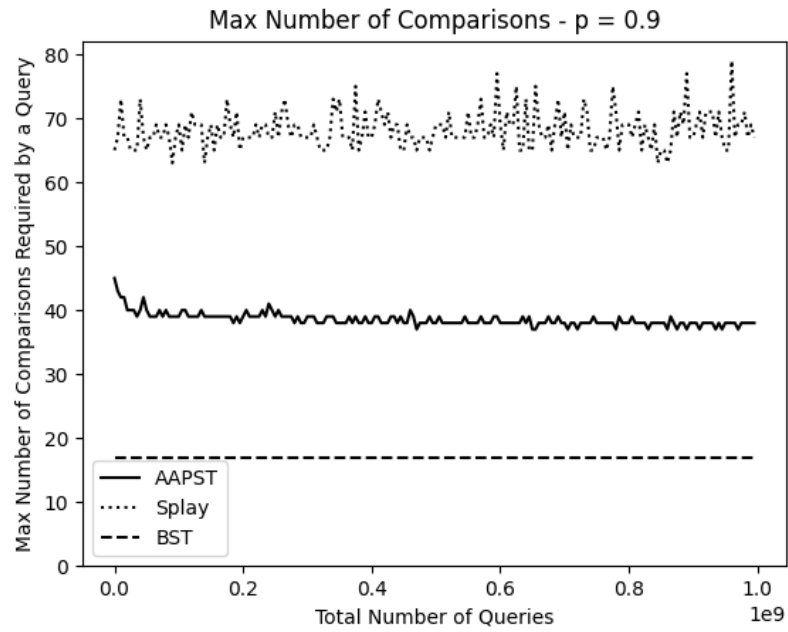


Figure 14: This figure shows the maximum number of key comparisons performed in a single query in a set of 5 million queries where 90% of the query keys are sampled according to an exponential distribution

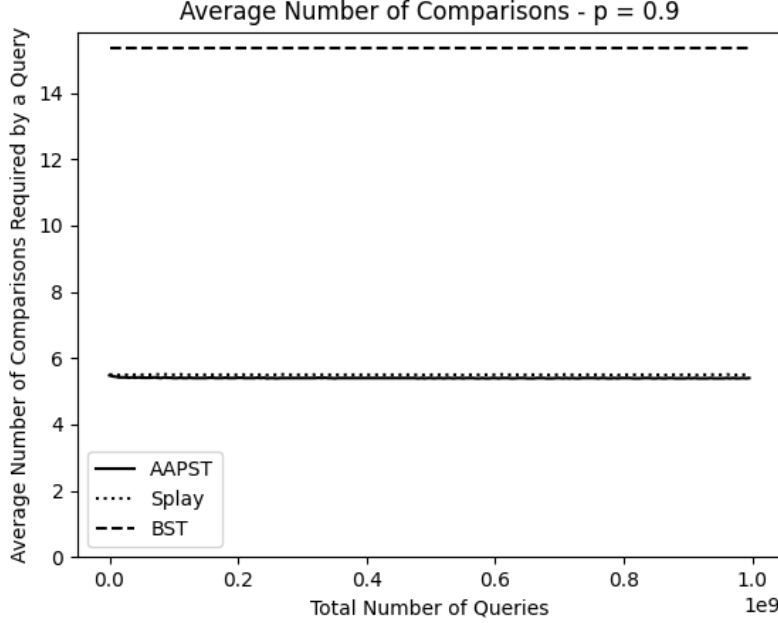


Figure 15: This figure shows the average number of key comparisons per query in a set of 5 million queries where 90% of the query keys are sampled according to an exponential distribution

6 Discussion

It is not unusual for the standard metrics applied for assessing algorithms in the literature to fail to capture salient characteristics of interest in important, though in some cases specialized, practical applications. The amortized performance characteristics of the splay tree are certainly of significant theoretical importance, but its practical use is highly constrained by the fact that most query-retrieval applications involve interactive or related real-time constraints. In this paper we have emphasized that some important applications, such as in reliability-critical space systems, include both fixed-bound performance constraints and a need to minimize overall computational costs so as to reduce energy consumption and/or unwanted heat generation. With this in mind, we adapted the classical priority search tree to create a new access-adaptive priority search tree (AAPST) which satisfies these joint performance goals.

Another contribution of this paper is an empirical examination of the performance properties of the AAPST in terms of key variables relating to the bounding of worst-case behavior and expected-case computational expenditures as determined by a measure of nonuniformity of the distribution of key accesses.

Our simulation results characterize the cases in which the AAPST can be expected to provide performance advantages over the splay tree and standard binary search trees.

In summary, our simulation results show that the AAPST offers comparable access-sensitive performance to the splay tree while strictly bounding the complexity of each operation. This is a property that is required for interactive applications that must impose strict constraints on the worst-case response time of each operation. Future work will examine finer-grain performance characteristics of the AAPST and their relevance to practical applications.

References

- [1] B. Allen and I. Munro, “Self-organizing search trees,” *Journal of the ACM*, 25 (4): 526–535, 1978.
- [2] D.E. Eckhahardt, “Time-Critical Multirate Scheduling Using Contemporary Real-Time Operating System Services,” NASA Technical Memorandum 85665, 1983. (<https://ntrs.nasa.gov/api/citations/19830027380/downloads/19830027380.pdf>)
- [3] J.W. Jordan, “Task Scheduling for a Real Time Multiprocessor,” NASA Technical Note D-5786, Electronics Research Center, 1970. (<https://ntrs.nasa.gov/api/citations/19700017975/downloads/19700017975.pdf>)
- [4] McCreight, Edward, “Priority search trees,” *SIAM Journal on Scientific Computing*, 14 (2): 257–276, 1985.
- [5] Sedgewick, Robert, “Balanced Trees,” *Algorithms*, Addison-Wesley, 1983.
- [6] Sleator, Daniel D.; Tarjan, Robert E., “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, 32 (3): 652–686. 1985.